OptiProductio project brief (v3)

Project title: Arduino-Based Real-Time WIP Monitoring and Bottleneck Detection

IDEA Lab Experiential Learning · Industrial Engineering and Management · Batch 2 · updated 1 November 2026 · Phase I review: Thursday 29 October 2026

> **How to read this brief.** This is the full working of our project in one place, so every teammate can explain any part of it. Each page ends with a box called “Terms on this page” that explains the jargon used on that page in plain words. Numbers in square brackets, like [3], point to the reference list near the end. Red blanks like **__ %** are results we have not measured yet; we fill them only after the runs. All prices are estimates, not quotes. The last section is a labelled 3D view of how the finished line will look on evaluation day.

1. The project in one page

Our official project title is “Arduino-Based Real-Time WIP Monitoring and Bottleneck Detection”. It sits under **our EL topic** (IEM, row 3 of the topic sheet), “Smart Production Workflow and Resource Optimization System”. The sheet asks us to monitor production count, cycle time, inventory and workflow, and to deliver a production efficiency dashboard.

What we are building. OptiProductio is a small production line on a table, with three hand-worked stations, a raw-material bin and a scrap tray. Cheap sensors on an Arduino UNO notice every part that leaves a station, every rejected part and every stop. The Arduino sends each of these as one short line of text over a USB cable to a laptop. A Python program saves the lines to a file, and a Streamlit dashboard turns them into the numbers a supervisor needs: good parts, scrap rate, cycle time per station, work-in-process (WIP), throughput, raw material left and an approximate OEE.

Why. Small factories, a large part of India’s economy [1], often count output by hand at the end of the shift, so a slow station or a pile of waiting parts is only noticed when someone walks past [4], [2]. Our prototype shows the same problems live, names the bottleneck, and warns when too many parts pile up inside the line, using the CONWIP idea of a fixed cap on WIP [5].

What the EL sheet asks for	How OptiProductio does it
Production count	An IR sensor at the exit of each station counts parts; a Reject button counts scrap
Cycle time	The Arduino stamps the time of every part; Python works out how long each station takes per part

TERMS ON THIS PAGE

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

Bottleneck: The one step that limits how fast the whole line can make good parts. *Like: The narrow neck of a bottle that sets how fast water pours.*

Production count: How many parts a station or the line has finished. *Like: Counting runs in a cricket match.*

Cycle time: How long one station works on one part, not counting time spent waiting for work. *Like: How long the barber actually spends on one haircut.*

Inventory: Material kept in stock waiting to be used, here the parts in the raw-material bin. *Like: The rice in your kitchen jar.*

Streamlit: A Python library that turns a script into a web page with charts and sliders. *Like: Instant noodles for dashboards: quick and simple.*

Scrap rate: Scrapped parts divided by all parts made. *Like: Out of 10 rotis, how many got burnt.*

Throughput: Good parts made per unit of time. *Like: Litres per minute coming out of a tap.*

OEE proxy: Our approximate OEE, because the ideal cycle time is our own estimate. *Like: A rough grade before the official marks come out.*

OEE (overall equipment effectiveness): One score: availability × performance × quality. *Like: A report-card grade combining attendance, speed and accuracy.*

Pile-up / queue: Parts waiting in front of a station because it cannot keep up. *Like: A queue at a single ticket counter.*

CONWIP / WIP cap: A fixed limit on how many parts may be inside the line; a new part enters only when one leaves. *Like: A club that lets one person in only when one comes out.*

IR obstacle sensor (FC-51): Shines invisible infrared light and notices when something close reflects it back. *Like: Like an automatic door that opens when you walk up to it.*

What the EL sheet asks for	How OptiProductio does it
Inventory	An ultrasonic sensor above the raw-material bin estimates how many parts are left; WIP is counted inside the line
Workflow monitoring	Breakdown and No-material buttons log stops with a reason; the dashboard shows parts waiting at each station
Production efficiency dashboard	A Streamlit web page on the laptop, refreshed every 2 seconds

What it is not. It is a prototype for learning and measuring, not a factory product. There is no machine learning, no motorised conveyor, no Wi-Fi board and no cloud. Parts are moved by hand. We are not claiming a patent, because software and algorithms on their own are generally not patentable in India [13]; our planned outcome is an open prototype and a short IEEE-format paper.

2. The problem: four bottlenecks and the pile-ups they cause

A **bottleneck** is the one step that limits how fast the whole line can make good parts, like the narrow neck of a bottle that decides how fast water pours out [15]. On a small manual line, four kinds of bottleneck keep turning up. Our line is built so that each one can be made to happen on purpose and then spotted by the system.

#	Bottleneck	What you see on the line	What it does to the other stations	How OptiProductio notices it	What we try as the fix
1	**Slow station** (Station 2, slow on purpose)	Parts queue in front of Station 2	Station 3 waits with nothing to do (starving)	Station 2 has the longest cycle time, and the “parts waiting” bar in front of it keeps growing	Move part of Station 2’s task to Station 1 (line balancing)
2	**Breakdown**	A station stops working	Parts pile up before it (blocking) and the stations after it starve	Breakdown button: the line goes red, the LCD shows BRKD, downtime is logged with reason “B” and availability drops	Fix it fast; the downtime-by-reason list shows how much time was lost
3	**Empty raw-material bin**	Station 1 has nothing to pick up	Every station after it starves in turn	The ultrasonic sensor sees the bin level fall; the dashboard says “refill now” at the reorder point; No-material button logs reason “M” if it runs out	Refill at the reorder point, before it is empty

TERMS ON THIS PAGE

Inventory: Material kept in stock waiting to be used, here the parts in the raw-material bin. *Like: The rice in your kitchen jar.*

Ultrasonic sensor (HC-SR04): Sends a click of sound too high to hear and times the echo to measure distance. *Like: A bat finding a wall by listening to its own squeak.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

Streamlit: A Python library that turns a script into a web page with charts and sliders. *Like: Instant noodles for dashboards: quick and simple.*

Bottleneck: The one step that limits how fast the whole line can make good parts. *Like: The narrow neck of a bottle that sets how fast water pours.*

Pile-up / queue: Parts waiting in front of a station because it cannot keep up. *Like: A queue at a single ticket counter.*

Starving: A station sits idle because no parts are reaching it. *Like: A cashier with no customers.*

Cycle time: How long one station works on one part, not counting time spent waiting for work. *Like: How long the barber actually spends on one haircut.*

Line balancing (rebalancing): Moving work between stations so they take about the same time. *Like: Sharing the bags so every hiker carries a similar load.*

Blocking: A station cannot pass its finished part on because the next step is full or stopped. *Like: A waiter holding plates because the table is not ready.*

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*

Downtime: Time the line is stopped, logged with a reason (B breakdown, M no material). *Like: Minutes a shop is shut during opening hours.*

Reorder point: The stock level at which you refill, so you do not run out while the refill arrives. *Like: Buying milk when one packet is left, not when it's gone.*

#	Bottleneck	What you see on the line	What it does to the other stations	How OptiProductio notices it	What we try as the fix
4	**Burst of rejects**	Many bad parts at inspection	Stations look busy, but good output drops; inspection gets overloaded	Reject button: scrap count and scrap rate rise, quality (and so OEE) falls	Find the cause at the station that made them

What a pile-up is and why it matters. A pile-up is WIP: parts that have entered the line but not left it. Parts pile up in front of whichever station is slowest, because the stations before it keep sending work faster than it can finish [11], [7]. Here is a worked example (made-up numbers, not a result): if Station 1 needs 6 s per part and Station 2 needs 12 s, Station 1 sends 10 parts a minute but Station 2 finishes only 5, so 5 extra parts pile up every minute. The line still makes only 5 parts a minute, because no part can get past Station 2 faster.

Why piled-up parts are bad, using Little's law. Little's law says average WIP = throughput × average flow time [7]. Throughput is fixed by the bottleneck, so every extra part waiting inside the line only makes each part take longer to get through. A pile of 10 parts in front of a station that does 5 a minute means a new part waits about 2 minutes before it is even touched.

The WIP cap. CONWIP sets a maximum number of parts allowed inside the line [5]. When WIP reaches the cap, our red LED lights and the buzzer beeps, and Station 1 must wait before releasing the next part. The line then holds only as much work as the bottleneck can handle, so flow time falls while throughput stays about the same, as long as the cap is not too low [5], [6], [9].

3. Bottlenecks that may pop up after we fix things

Fixing one bottleneck does not leave a line with no bottleneck. These are the ones we expect to meet once the system works, and we plan for each.

On the line itself

- **The bottleneck moves.** After we rebalance Station 2, Station 1 (which now has extra work) or Station 3 may become the slowest. This is normal: the Theory of Constraints says a line always has one constraint, and improving it moves the constraint somewhere else [14]. The dashboard will simply name the new slowest station, and we report the new cycle times in the results.
- **WIP cap set too low.** If the cap is too tight, the bottleneck station runs out of parts to work on and throughput drops [5], [15]. That is why we test caps of 2, 4 and 6 and not just one value.
- **The release point.** Under a cap, Station 1 has to watch the LED and stop releasing. If the operator reacts late, WIP goes one or two above the cap. We note this in the run sheet.

TERMS ON THIS PAGE

Bottleneck: The one step that limits how fast the whole line can make good parts. *Like: The narrow neck of a bottle that sets how fast water pours.*

Scrap rate: Scrapped parts divided by all parts made. *Like: Out of 10 rotis, how many got burnt.*

Quality: Good parts divided by all parts made. *Like: Accuracy: how many answers were right.*

OEE (overall equipment effectiveness): One score: availability × performance × quality. *Like: A report-card grade combining attendance, speed and accuracy.*

Pile-up / queue: Parts waiting in front of a station because it cannot keep up. *Like: A queue at a single ticket counter.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

Little's law: Average WIP = throughput × average flow time. *Like: More people in the queue means a longer wait if the counter's speed stays the same.*

Throughput: Good parts made per unit of time. *Like: Litres per minute coming out of a tap.*

Flow time (lead time): How long one part takes from entering the line to leaving it, including waiting. *Like: Your total time at the barber's, queue included.*

CONWIP / WIP cap: A fixed limit on how many parts may be inside the line; a new part enters only when one leaves. *Like: A club that lets one person in only when one comes out.*

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

Active buzzer: A buzzer with its own tone circuit, so switching on 5 V is enough to make it beep. *Like: A doorbell that rings by itself once powered.*

Shifting bottleneck: When the old bottleneck is fixed and another step becomes the slowest. *Like: Unblock one lane of traffic and the jam moves to the next junction.*

Line balancing (rebalancing): Moving work between stations so they take about the same time. *Like: Sharing the bags so every hiker carries a similar load.*

Theory of Constraints: The idea that every system has one constraint that limits it, and you improve the system by improving that constraint first. *Like: A chain is only as strong as its weakest link.*

Cycle time: How long one station works on one part, not counting time spent waiting for work. *Like: How long the barber actually spends on one haircut.*

- **Inspection overload.** A burst of rejects makes Station 3 slower (each bad part still needs checking and a button press), so Station 3 can briefly become the bottleneck.
- **Raw material runs out faster.** Once throughput goes up, the bin empties sooner. The reorder point (the slider in the dashboard) has to cover the parts used while someone refills the bin, plus a small safety stock [16].

In the measuring system

- **IR gate speed.** The sketch ignores the same gate for 400 ms after a count, so two parts closer than 0.4 s apart at one gate count as one (at most about 2.5 parts a second per gate). Our hand-worked stations are far slower than this, so it should not matter, and the hand-tally check (PO1) will show it if it does.
- **Human button timing.** Downtime starts when someone presses Breakdown, not when the problem started. A late press makes downtime look shorter. We agree beforehand who presses which button.
- **One Breakdown button for the whole line.** The button says the line stopped, not which station. In staged tests we write down which station we stopped; a per-station button is a possible later upgrade.
- **Station 1 is assumed never starved** in the cycle-time maths. When the bin is empty, Station 1's cycle time looks longer than it really is. Using the median (the middle value) keeps a few long waits from spoiling it.
- **USB and laptop.** The Arduino sends at 9600 baud, roughly 960 characters a second, and our lines are about 30 characters, so about 30 lines a second fit; we send fewer than 5. The dashboard re-reads the whole CSV every 2 s, which is fine for a 10-minute run of a few hundred rows.

In our own build plan

- Parts arriving late (ask the IDEA Lab first; order the rest the same day).
- The LCD staying blank because its address is 0x3F instead of 0x27.
- "Port busy" errors when the Arduino IDE Serial Monitor is still open while the logger runs.
- Too little time for the experiments: the must-haves are the IR counters, buttons, LEDs and the dashboard. The LCD and bin sensor are the first things to drop if we fall behind.

TERMS ON THIS PAGE

Bottleneck: The one step that limits how fast the whole line can make good parts. *Like: The narrow neck of a bottle that sets how fast water pours.*

Throughput: Good parts made per unit of time. *Like: Litres per minute coming out of a tap.*

Reorder point: The stock level at which you refill, so you do not run out while the refill arrives. *Like: Buying milk when one packet is left, not when it's gone.*

Safety stock: A little extra stock kept for surprises. *Like: A spare pen in your pencil box.*

IR obstacle sensor (FC-51): Shines invisible infrared light and notices when something close reflects it back. *Like: Like an automatic door that opens when you walk up to it.*

Sketch (Arduino program): The program we upload to the Arduino; it runs in a loop forever. *Like: A recipe the cook repeats again and again.*

Downtime: Time the line is stopped, logged with a reason (B breakdown, M no material). *Like: Minutes a shop is shut during opening hours.*

Starving: A station sits idle because no parts are reaching it. *Like: A cashier with no customers.*

Cycle time: How long one station works on one part, not counting time spent waiting for work. *Like: How long the barber actually spends on one haircut.*

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

Baud rate (9600): How fast serial data is sent, in signal changes per second; 9600 baud is about 960 characters a second. *Like: The speaking speed both sides agree on.*

CSV file: A plain text table where each line is a row and commas separate the columns. *Like: A spreadsheet saved as plain text.*

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*

I2C address (0x27 / 0x3F): The number a device answers to on the I2C wires. *Like: A house number on a shared street.*

Arduino IDE: The free program used to write and upload sketches to the Arduino. *Like: A word processor for Arduino code.*

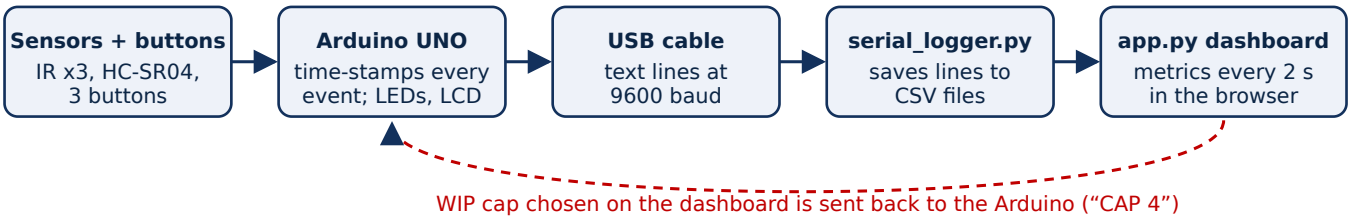
USB serial / COM port: The Arduino sends text to the laptop over the USB cable; the laptop sees it as a port such as COM3. *Like: A phone line between the Arduino and the laptop.*

Serial Monitor: A window in the Arduino IDE that shows the text the Arduino sends. *Like: A peephole into the phone line.*

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

4. How one part moves through the line

Follow one wooden block from the bin to the good-parts tray. Each step says what the person does, what the Arduino does, and what changes on the dashboard.



1. **Release.** The Station 1 operator looks at the LEDs. Green means WIP is under the cap and the line is running, so they take one block from the raw-material bin. The ultrasonic sensor sees the pile drop a little; once a second the Arduino turns that distance into “parts left in bin”.
2. **Station 1 (Cut).** The operator does a small task, for example sticking a label on the block, then pushes it through the cardboard gate at the station exit. The IR sensor in the gate sees it. The Arduino adds 1 to Station 1’s count and sends `E,41300,PASS,1` (event, time in milliseconds, what happened, which station). WIP goes up by 1, because a new part is now inside the line.
3. **Waiting.** The block sits in front of Station 2 until that operator is free. This waiting is the pile-up we want to see.
4. **Station 2 (Drill, slow on purpose).** A longer task, for example stacking three caps and putting a rubber band round them. Through gate 2: `E,…,PASS,2`.
5. **Station 3 (Inspect).** The inspector checks the part. A good part goes through gate 3 into the good-parts tray: `E,…,PASS,3`, the good count goes up and WIP goes down by 1. A bad part goes into the scrap tray and the inspector presses the red Reject button: `E,…,SCRAP,3`, scrap goes up and WIP goes down by 1.
6. **Every second** the Arduino also sends a status line, refreshes the LCD, sets the LEDs, and beeps if WIP is at or over the cap.
7. **On the laptop**, the logger writes each line to a CSV file within a fraction of a second, and the dashboard re-reads the file every 2 seconds and redraws every number and chart.

5. The hardware, one component at a time

Everything runs from the Arduino’s 5 V supply over the USB cable, so no extra power supply is needed. Pin numbers match the sketch `flowloop_line.ino`. Prices are our estimates from typical Robu.in and SP Road rates.

TERMS ON THIS PAGE

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

Ultrasonic sensor (HC-SR04): Sends a click of sound too high to hear and times the echo to measure distance. *Like: A bat finding a wall by listening to its own squeak.*

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*

Baud rate (9600): How fast serial data is sent, in signal changes per second; 9600 baud is about 960 characters a second. *Like: The speaking speed both sides agree on.*

pyserial: A Python library that reads and writes the USB serial port. *Like: The laptop's ear on the phone line.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

CONWIP / WIP cap: A fixed limit on how many parts may be inside the line; a new part enters only when one leaves. *Like: A club that lets one person in only when one comes out.*

Blocking: A station cannot pass its finished part on because the next step is full or stopped. *Like: A waiter holding plates because the table is not ready.*

IR obstacle sensor (FC-51): Shines invisible infrared light and notices when something close reflects it back. *Like: Like an automatic door that opens when you walk up to it.*

Pile-up / queue: Parts waiting in front of a station because it cannot keep up. *Like: A queue at a single ticket counter.*

CSV file: A plain text table where each line is a row and commas separate the columns. *Like: A spreadsheet saved as plain text.*

Pin (digital pin, D2 to D12): A metal leg on the Arduino that can read 5 V as HIGH and 0 V as LOW, or switch 5 V on and off. *Like: A light switch the Arduino can flip or check.*

Sketch (Arduino program): The program we upload to the Arduino; it runs in a loop forever. *Like: A recipe the cook repeats again and again.*

5.1 Arduino UNO R3 (the brain)

- **What it does here:** reads all the sensors and buttons thousands of times a second, stamps each event with the time, drives the LEDs, buzzer and LCD, and sends everything to the laptop over USB [17].
- **Why this and not something else:** it is standard IDEA Lab kit, has enough pins for our 10 inputs and outputs, and needs no Wi-Fi setup. A Wi-Fi board (ESP32) would add networking problems without teaching any extra IEM idea.
- **How it works:** a small computer on one chip. It runs one program (the sketch) in a loop forever. Its pins can read a voltage as HIGH (5 V) or LOW (0 V), or switch 5 V on and off.
- **Connection:** USB A-to-B cable to the laptop. **Cost:** ₹450–650 (a CH340 clone is fine), or borrowed from the lab.

5.2 IR obstacle sensors, FC-51, three of them (the counters)

- **What they do here:** one sits in a gate at the exit of each station and counts every part that leaves it.
- **Why they are there:** production count and cycle time both come from knowing when each part left each station. Three gates let us see the time spent at each station separately, which is how we find which station is the bottleneck.
- **Why IR and not a camera or a switch:** about ₹50 each, a simple yes/no signal, nothing to touch or wear out, and no image processing.
- **How they work:** an IR LED shines invisible light forward and a receiver looks for the reflection. When a block passes within about 3–5 cm, light bounces back and the OUT pin drops from HIGH to LOW. The sketch counts each HIGH-to-LOW drop as one part, then ignores that gate for 400 ms so one part is not counted twice.
- **Pins:** Station 1 exit to D2, Station 2 exit to D3, Station 3 exit (good parts) to D4. VCC to 5 V, GND to GND.
- **Setting up:** turn the blue screw so the sensor triggers at 3–5 cm and not further. Build each gate from two cardboard strips so every block passes close to the sensor. Test with the real blocks: dark or shiny parts reflect differently.

5.3 HC-SR04 ultrasonic sensor (the bin level)

- **What it does here:** looks straight down into the raw-material bin and estimates how many parts are left.

TERMS ON THIS PAGE

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

Active buzzer: A buzzer with its own tone circuit, so switching on 5 V is enough to make it beep. *Like: A doorbell that rings by itself once powered.*

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*

Pin (digital pin, D2 to D12): A metal leg on the Arduino that can read 5 V as HIGH and 0 V as LOW, or switch 5 V on and off. *Like: A light switch the Arduino can flip or check.*

Sketch (Arduino program): The program we upload to the Arduino; it runs in a loop forever. *Like: A recipe the cook repeats again and again.*

HIGH / LOW: The two states a digital pin can be in: about 5 V (HIGH) or about 0 V (LOW). *Like: Lights on or lights off.*

IR obstacle sensor (FC-51): Shines invisible infrared light and notices when something close reflects it back. *Like: Like an automatic door that opens when you walk up to it.*

Production count: How many parts a station or the line has finished. *Like: Counting runs in a cricket match.*

Cycle time: How long one station works on one part, not counting time spent waiting for work. *Like: How long the barber actually spends on one haircut.*

Bottleneck: The one step that limits how fast the whole line can make good parts. *Like: The narrow neck of a bottle that sets how fast water pours.*

Blocking: A station cannot pass its finished part on because the next step is full or stopped. *Like: A waiter holding plates because the table is not ready.*

5 V and GND: 5 V is the supply voltage; GND (ground) is the 0 V return path every part must also connect to. *Like: The tap and the drain: water needs both to flow.*

Ultrasonic sensor (HC-SR04): Sends a click of sound too high to hear and times the echo to measure distance. *Like: A bat finding a wall by listening to its own squeak.*

- **Why it is there:** inventory is one of the four things the EL sheet asks us to monitor, and an empty bin is one of our four bottlenecks. With a live bin level the dashboard can warn at the reorder point, before the line starves.
- **How it works:** it sends a short burst of sound too high for us to hear and times the echo from the top of the pile. Distance in cm = echo time in microseconds ÷ 58 [18]. Fewer parts means the top of the pile is further away. The sketch turns the distance into parts: parts = (EMPTY_CM − distance) ÷ PART_CM, rounded.
- **Pins:** TRIG to D11, ECHO to D12, VCC to 5 V, GND to GND. **Cost:** ₹70–100.
- **Setting up:** mount it on a stand about 20 cm above the bin floor, pointing straight down. Measure the empty-bin distance (EMPTY_CM, default 20) and the height one part adds to the pile (PART_CM, default 2) with a ruler and put both in the sketch. Keep the parts stacked level; an uneven heap gives a rough reading.

5.4 Three push buttons (the human inputs)

Button	Pin	What one press does	Why we need it
Breakdown (yellow)	D5	First press: line stopped, reason B. Second press: running again	Records downtime and its reason; needed for availability and OEE, and for bottleneck 2
No material (blue)	D6	Same toggle, reason M	Records stops caused by an empty bin (bottleneck 3)
Reject (red, at Station 3)	D7	One press = one scrapped part	Counts scrap for scrap rate and quality (bottleneck 4)

- **How they work:** each button connects its pin to GND when pressed. The sketch switches on the UNO’s built-in pull-up resistor, so the pin reads HIGH normally and LOW when pressed, and no extra resistor is needed. Metal contacts flicker for a few milliseconds when pressed, so the sketch ignores further changes for 200 ms (debouncing).
- **Why buttons and not sensors for these:** a machine cannot easily tell a breakdown from a pause, or a good part from a bad one. In real factories operators also log stop reasons by hand on a terminal. **Cost:** ₹5–30 each.

TERMS ON THIS PAGE

Inventory: Material kept in stock waiting to be used, here the parts in the raw-material bin. *Like: The rice in your kitchen jar.*

Bottleneck: The one step that limits how fast the whole line can make good parts. *Like: The narrow neck of a bottle that sets how fast water pours.*

Reorder point: The stock level at which you refill, so you do not run out while the refill arrives. *Like: Buying milk when one packet is left, not when it's gone.*

Starving: A station sits idle because no parts are reaching it. *Like: A cashier with no customers.*

Sketch (Arduino program): The program we upload to the Arduino; it runs in a loop forever. *Like: A recipe the cook repeats again and again.*

Pin (digital pin, D2 to D12): A metal leg on the Arduino that can read 5 V as HIGH and 0 V as LOW, or switch 5 V on and off. *Like: A light switch the Arduino can flip or check.*

5 V and GND: 5 V is the supply voltage; GND (ground) is the 0 V return path every part must also connect to. *Like: The tap and the drain: water needs both to flow.*

Downtime: Time the line is stopped, logged with a reason (B breakdown, M no material). *Like: Minutes a shop is shut during opening hours.*

Availability: Run time divided by planned time. *Like: Attendance: how much of the lesson you were present for.*

OEE (overall equipment effectiveness): One score: availability × performance × quality. *Like: A report-card grade combining attendance, speed and accuracy.*

Scrap rate: Scrapped parts divided by all parts made. *Like: Out of 10 rotis, how many got burnt.*

Quality: Good parts divided by all parts made. *Like: Accuracy: how many answers were right.*

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

Pull-up resistor (INPUT_PULLUP): A built-in resistor that holds a pin at HIGH until a button pulls it to GND. *Like: A spring that keeps a door shut until you push it.*

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

HIGH / LOW: The two states a digital pin can be in: about 5 V (HIGH) or about 0 V (LOW). *Like: Lights on or lights off.*

Debounce: Ignoring the quick on-off flicker of a switch's metal contacts so one press counts once. *Like: Waiting for a dropped ball to stop bouncing before you count it as landed.*

5.5 Green and red LEDs with 220 Ω resistors, and an active buzzer (the andon)

- **What they do here:** green (D8) means the line is running and WIP is under the cap, so it is OK to release a part. Red (D9) means WIP is at or over the cap, or the line is stopped. The buzzer (D10) gives a short beep every second while WIP is at or over the cap.
- **Why they are there:** this is an andon, the signal light used on Toyota-style lines so that everyone on the floor sees a problem at once [19]. It is how the WIP cap is actually obeyed: the Station 1 operator does not need to look at the laptop.
- **How they work:** the pin switches 5 V on or off. The 220 Ω resistor limits the current so the LED does not burn out. An active buzzer has its own tone circuit, so 5 V on its pin is enough to make it beep.
- **Wiring:** LED long leg through the resistor to the pin, short leg to GND; buzzer + to D10, − to GND. **Cost:** about ₹35 for all three parts.

5.6 16x2 LCD with I2C backpack (the line-side display)

- **What it does here:** shows the key numbers at the line itself, refreshed every second, so operators do not have to walk to the laptop.

LCD row	Example	Meaning
Row 1	`G23 S2 WIP5 !`	23 good parts, 2 scrapped, 5 parts inside the line, ! = at or over the cap
Row 2	`B12 RUN CAP5`	12 parts left in the bin, line running (BRKD = breakdown, NMAT = no material), WIP cap 5

- **Why I2C:** a plain LCD needs 6 or more pins; the I2C backpack needs only two (SDA to A4, SCL to A5), which leaves pins free for our sensors.
- **Setting up:** install the LiquidCrystal I2C library in the Arduino IDE. If the screen is blank, turn the small contrast screw on the backpack, then try address 0x3F in place of 0x27. **Cost:** ₹180–250.

5.7 Breadboard, jumper wires, base board, gates and trays

- **Breadboard (830 points):** lets us connect everything without soldering. The red rail carries 5 V and the blue rail carries GND to every part. ₹80–120.
- **Jumper wires:** male-male and male-female, about 40 of each. ₹100–150. Tape them down so a hand moving a part cannot pull one out.

TERMS ON THIS PAGE

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

Active buzzer: A buzzer with its own tone circuit, so switching on 5 V is enough to make it beep. *Like: A doorbell that rings by itself once powered.*

Andon: A signal light that tells everyone on the floor about a problem straight away. *Like: A traffic light for the line.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

CONWIP / WIP cap: A fixed limit on how many parts may be inside the line; a new part enters only when one leaves. *Like: A club that lets one person in only when one comes out.*

Pin (digital pin, D2 to D12): A metal leg on the Arduino that can read 5 V as HIGH and 0 V as LOW, or switch 5 V on and off. *Like: A light switch the Arduino can flip or check.*

5 V and GND: 5 V is the supply voltage; GND (ground) is the 0 V return path every part must also connect to. *Like: The tap and the drain: water needs both to flow.*

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*

I2C (SDA, SCL): A way for chips to talk over just two wires, one for data (SDA) and one for the clock (SCL). *Like: Two people sharing one walkie-talkie channel and taking turns.*

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

Arduino IDE: The free program used to write and upload sketches to the Arduino. *Like: A word processor for Arduino code.*

I2C address (0x27 / 0x3F): The number a device answers to on the I2C wires. *Like: A house number on a shared street.*

Breadboard: A board with hidden metal strips for connecting parts by pushing in wires, no soldering. *Like: LEGO baseplate for electronics.*

- **The line itself:** an A2 foam board or cardboard base, three marked station pads, three cardboard IR gates, a raw-material bin (a small box), a good-parts tray and a scrap tray. Parts are 30 identical wooden blocks, bottle caps or LEGO bricks. ₹50–250 in total.
- **Laptop:** runs the logger and the dashboard. Any laptop with Python 3 and a free USB port.

Total cost: about ₹1,400–1,900 if we buy everything, or about ₹600–900 if the lab lends the UNO, breadboard and wires (our estimate).

5.8 Wiring table

Part	Part pin to UNO pin	Notes
All parts	5V to red rail, GND to blue rail	Do this first
IR sensor, Station 1 exit	OUT to D2	LOW when a part is in front of it
IR sensor, Station 2 exit	OUT to D3	
IR sensor, Station 3 exit (good parts)	OUT to D4	
Breakdown button	one leg to D5, other to GND	Toggle: press to stop, press again to run
No material button	one leg to D6, other to GND	Toggle
Reject button	one leg to D7, other to GND	One press = one scrap
Green LED	long leg via 220 Ω to D8, short leg to GND	
Red LED	long leg via 220 Ω to D9, short leg to GND	
Active buzzer	+ to D10, − to GND	
HC-SR04	TRIG to D11, ECHO to D12	About 20 cm above the bin floor
16x2 LCD (I2C)	SDA to A4, SCL to A5	Address 0x27 or 0x3F

6. The software: what each program does

There are four pieces of code in our project folder. All of them are already written; the sketch has been syntax-checked but not yet run on a real UNO, so expect a small fix or two.

TERMS ON THIS PAGE

IR obstacle sensor (FC-51): Shines invisible infrared light and notices when something close reflects it back. *Like: Like an automatic door that opens when you walk up to it.*

Blocking: A station cannot pass its finished part on because the next step is full or stopped. *Like: A waiter holding plates because the table is not ready.*

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

Breadboard: A board with hidden metal strips for connecting parts by pushing in wires, no soldering. *Like: LEGO baseplate for electronics.*

Pin (digital pin, D2 to D12): A metal leg on the Arduino that can read 5 V as HIGH and 0 V as LOW, or switch 5 V on and off. *Like: A light switch the Arduino can flip or check.*

5 V and GND: 5 V is the supply voltage; GND (ground) is the 0 V return path every part must also connect to. *Like: The tap and the drain: water needs both to flow.*

HIGH / LOW: The two states a digital pin can be in: about 5 V (HIGH) or about 0 V (LOW). *Like: Lights on or lights off.*

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

Active buzzer: A buzzer with its own tone circuit, so switching on 5 V is enough to make it beep. *Like: A doorbell that rings by itself once powered.*

Ultrasonic sensor (HC-SR04): Sends a click of sound too high to hear and times the echo to measure distance. *Like: A bat finding a wall by listening to its own squeak.*

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*

I2C (SDA, SCL): A way for chips to talk over just two wires, one for data (SDA) and one for the clock (SCL). *Like: Two people sharing one walkie-talkie channel and taking turns.*

I2C address (0x27 / 0x3F): The number a device answers to on the I2C wires. *Like: A house number on a shared street.*

Sketch (Arduino program): The program we upload to the Arduino; it runs in a loop forever. *Like: A recipe the cook repeats again and again.*

6.1 The Arduino sketch, `flowloop_line.ino`

The `loop()` does five jobs over and over: read the three IR sensors, read the three buttons, listen for a new WIP cap from the laptop, set the LEDs and buzzer, and once a second send a status line and update the LCD. WIP is worked out on the Arduino itself as: parts out of Station 1 − good parts out of Station 3 − scrapped parts.

It sends two kinds of text line:

Line	Example	Meaning
Event	`E,41300,PASS,2`	At 41.3 s a part left Station 2. Other types: `SCRAP,3`, `DOWN,B` (breakdown), `UP,B` (running again), `DOWN,M` / `UP,M` (no material)
Status	`S,42000,25,24,22,1,2,12,R,5`	Time, counts at S1, S2, S3, scrap, WIP, parts in bin, state (R running, B breakdown, M no material), WIP cap

The laptop can send back `CAP 4`, and the sketch changes its WIP cap straight away.

Settings at the top of the sketch: `wipCap` (default 5), `EMPTY_CM` (20), `PART_CM` (2), `IR_GAP_MS` (400 ms, ignore repeat counts) and `BTN_GAP_MS` (200 ms debounce).

6.2 The logger, `serial_logger.py`

Run with `python serial_logger.py COM3` (on Mac or Linux, a port like `/dev/ttyACM0`). It opens the USB port with the pyserial library, waits 2 s (the UNO restarts when the port opens), and then reads line by line. Event lines go into `data/live_events.csv` with the columns t, event, station, reason, source. Status lines go into `data/live_status.csv`. It also watches `data/cap.txt`: when the dashboard writes a new cap there, the logger sends `CAP n` to the Arduino. Close the Arduino IDE's Serial Monitor first, because only one program can use the port at a time.

6.3 The metrics, `flowloop_metrics.py`

This file turns the list of events into IEM numbers using the pandas library. Each formula below is exactly what the code does.

Number	How it is worked out	What it tells us
Production count	Number of PASS events at each station; good parts = passes at Station 3	How much each station has done
Scrap rate	scrap ÷ (good + scrap)	Share of parts wasted

TERMS ON THIS PAGE

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

Sketch (Arduino program): The program we upload to the Arduino; it runs in a loop forever. *Like: A recipe the cook repeats again and again.*

IR obstacle sensor (FC-51): Shines invisible infrared light and notices when something close reflects it back. *Like: Like an automatic door that opens when you walk up to it.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

CONWIP / WIP cap: A fixed limit on how many parts may be inside the line; a new part enters only when one leaves. *Like: A club that lets one person in only when one comes out.*

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

Active buzzer: A buzzer with its own tone circuit, so switching on 5 V is enough to make it beep. *Like: A doorbell that rings by itself once powered.*

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*

Debounce: Ignoring the quick on-off flicker of a switch's metal contacts so one press counts once. *Like: Waiting for a dropped ball to stop bouncing before you count it as landed.*

pyserial: A Python library that reads and writes the USB serial port. *Like: The laptop's ear on the phone line.*

USB serial / COM port: The Arduino sends text to the laptop over the USB cable; the laptop sees it as a port such as COM3. *Like: A phone line between the Arduino and the laptop.*

Arduino IDE: The free program used to write and upload sketches to the Arduino. *Like: A word processor for Arduino code.*

Serial Monitor: A window in the Arduino IDE that shows the text the Arduino sends. *Like: A peephole into the phone line.*

pandas: A Python library for working with tables of data. *Like: Excel, but driven by code.*

Production count: How many parts a station or the line has finished. *Like: Counting runs in a cricket match.*

Scrap rate: Scrapped parts divided by all parts made. *Like: Out of 10 rotis, how many got burnt.*

Number	How it is worked out	What it tells us
Cycle time per station	Median of finish(k) − max(arrive(k), finish(k−1)) over all parts k, matched first in, first out	How long a station actually works on one part, leaving out time it spent waiting for work
Bottleneck	The station with the largest cycle time	Which station limits the line
Parts waiting	Count leaving the station before − count leaving this station (minus scrap at Station 3)	Size of the pile-up at each station
WIP now, average WIP	+1 when a part leaves Station 1, −1 when it leaves Station 3 or is scrapped; average weighted by time	How much work is stuck inside the line
Throughput	Good parts ÷ planned time, shown per minute	Output rate
Flow time (measured)	Exit time − Station 1 time for each part, first in, first out, then averaged	How long one part takes to cross the line
Flow time (Little's law)	Average WIP ÷ throughput [7]	A cross-check: should be close to the measured flow time
OEE (approximate)	Availability × Performance × Quality [3], [10]	One overall efficiency score
Takt time	Planned time ÷ customer demand	The pace the line must keep to meet demand

Why cycle time needs that formula. If we just took the gap between two parts leaving Station 3, Station 3 would look as slow as Station 2, because it spends most of its time waiting for Station 2. Taking away the waiting (a station can start part k only when part k has arrived and part k−1 is finished) gives each station's own working time, so the slow one stands out.

The OEE parts. Availability = run time ÷ planned time, where run time is planned time minus logged downtime. Performance = (ideal cycle time × parts made) ÷ run time, capped at 100%. Quality = good ÷ (good + scrap). We call our OEE approximate (an "OEE proxy") because the ideal cycle time, **__ s**, is our own estimate, set by the team after timing the stations.

Bosch mode. `bosch_to_events()` reads a slice (first 50,000 rows) of the public Bosch production-line data from Kaggle [12], picks the three busiest stations on line L3, and turns each part's time at those stations into the same PASS and SCRAP events (a failed quality check becomes SCRAP). The same dashboard then works on real factory data. Bosch did not publish its time unit; a community estimate is that 0.01 units is about 6 minutes [8], so we use the Bosch data only for comparing stations with each other, not for exact seconds.

TERMS ON THIS PAGE

Cycle time: How long one station works on one part, not counting time spent waiting for work. *Like: How long the barber actually spends on one haircut.*

FIFO (first in, first out): Parts are handled in the order they arrived. *Like: A bus queue.*

Bottleneck: The one step that limits how fast the whole line can make good parts. *Like: The narrow neck of a bottle that sets how fast water pours.*

Pile-up / queue: Parts waiting in front of a station because it cannot keep up. *Like: A queue at a single ticket counter.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

Time-weighted average: An average where each value counts for as long as it lasted. *Like: Average speed on a trip: an hour at 60 counts more than a minute at 100.*

Throughput: Good parts made per unit of time. *Like: Litres per minute coming out of a tap.*

Flow time (lead time): How long one part takes from entering the line to leaving it, including waiting. *Like: Your total time at the barber's, queue included.*

Little's law: Average WIP = throughput × average flow time. *Like: More people in the queue means a longer wait if the counter's speed stays the same.*

OEE (overall equipment effectiveness): One score: availability × performance × quality. *Like: A report-card grade combining attendance, speed and accuracy.*

OEE proxy: Our approximate OEE, because the ideal cycle time is our own estimate. *Like: A rough grade before the official marks come out.*

Availability: Run time divided by planned time. *Like: Attendance: how much of the lesson you were present for.*

Performance: How fast the line ran compared with its ideal cycle time. *Like: Speed: how many questions you answered in the time you were there.*

Quality: Good parts divided by all parts made. *Like: Accuracy: how many answers were right.*

Takt time: Available time divided by customer demand: the pace the line must keep. *Like: A metronome that sets the beat the line must follow.*

Downtime: Time the line is stopped, logged with a reason (B breakdown, M no material). *Like: Minutes a shop is shut during opening hours.*

Bosch dataset: Public data from a real Bosch production line, shared on the Kaggle website in 2016. *Like: A real factory's diary, made public.*

6.4 The dashboard, `app.py`

Run with `streamlit run app.py`; it opens in the browser and redraws itself every 2 seconds.

Why Streamlit. We write it in plain Python, every chart is one line of code, it runs on the laptop with no server or internet, and it is free [20]. Building the same page in HTML and JavaScript would take far longer and teach no extra IEM idea. For the look and feel, our style reference is **Velotic (velotic.com)**, GE Vernova's industrial software site, which Rohan picked as the benchmark for clean front-end work: lots of white space, few colours, and one strong alert colour.

What is on screen, top to bottom.

- **Sidebar (settings):** data source (live Arduino line or Bosch sample), ideal cycle time, customer demand, planned run time, a WIP-cap slider (sent to the Arduino) and a reorder-point slider for the bin.
- **Top row, five numbers:** good parts, scrap rate, throughput per minute, WIP now (with the cap), OEE.
- **WIP banner:** green "OK to release the next part" or red "Stop releasing parts into Station 1".
- **Cycle time per station:** a bar chart, a line naming the bottleneck station and its seconds per part, and the takt time to compare against.
- **Parts waiting in front of each station:** a bar chart that shows where the pile-up is.
- **WIP over time:** a line chart; it climbs when parts pile up and flattens under a cap.
- **OEE breakdown:** availability × performance × quality, and downtime in seconds by reason (B, M).
- **Little's law check:** predicted flow time beside measured flow time.
- **Raw-material bin:** parts left, and a red "refill now" message at or below the reorder point.

Still to add (Phase II). Today the dashboard shows the signal for each of the four bottlenecks in different panels, but it does not yet print one plain answer. PO3 needs it to name the cause, so we will add a small "What is holding the line up?" box with these rules, checked in this order: line state B means breakdown; state M or bin at zero means empty bin; more than a set share of the last few parts scrapped means reject burst; otherwise the station with the longest cycle time and a growing queue is the slow station. The thresholds are still to be chosen from our trial runs.

TERMS ON THIS PAGE

Streamlit: A Python library that turns a script into a web page with charts and sliders. *Like: Instant noodles for dashboards: quick and simple.*

Velotic: GE Vernova's industrial software brand; Rohan's chosen reference for clean front-end design. *Like: The design 'mood board' we copy the feel of.*

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

Bosch dataset: Public data from a real Bosch production line, shared on the Kaggle website in 2016. *Like: A real factory's diary, made public.*

Cycle time: How long one station works on one part, not counting time spent waiting for work. *Like: How long the barber actually spends on one haircut.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

CONWIP / WIP cap: A fixed limit on how many parts may be inside the line; a new part enters only when one leaves. *Like: A club that lets one person in only when one comes out.*

Scrap rate: Scrapped parts divided by all parts made. *Like: Out of 10 rotis, how many got burnt.*

Throughput: Good parts made per unit of time. *Like: Litres per minute coming out of a tap.*

OEE (overall equipment effectiveness): One score: availability × performance × quality. *Like: A report-card grade combining attendance, speed and accuracy.*

Bottleneck: The one step that limits how fast the whole line can make good parts. *Like: The narrow neck of a bottle that sets how fast water pours.*

Takt time: Available time divided by customer demand: the pace the line must keep. *Like: A metronome that sets the beat the line must follow.*

Pile-up / queue: Parts waiting in front of a station because it cannot keep up. *Like: A queue at a single ticket counter.*

Availability: Run time divided by planned time. *Like: Attendance: how much of the lesson you were present for.*

Performance: How fast the line ran compared with its ideal cycle time. *Like: Speed: how many questions you answered in the time you were there.*

Quality: Good parts divided by all parts made. *Like: Accuracy: how many answers were right.*

Downtime: Time the line is stopped, logged with a reason (B breakdown, M no material). *Like: Minutes a shop is shut during opening hours.*

Little's law: Average WIP = throughput × average flow time. *Like: More people in the queue means a longer wait if the counter's speed stays the same.*

Flow time (lead time): How long one part takes from entering the line to leaving it, including waiting. *Like: Your total time at the barber's, queue included.*

Reorder point: The stock level at which you refill, so you do not run out while the refill arrives. *Like: Buying milk when one packet is left, not when it's gone.*

7. Building it, step by step

Each step ends with a check. Do not move on until the check passes, so that when something breaks we know it is in the last thing we added.

Step	What to do	Check before moving on
1. Parts and software	Ask the IDEA Lab what they can lend; buy the rest. Install the Arduino IDE, the LiquidCrystal I2C library, Python 3, and `pip install pyserial pandas streamlit`	`python -c "import serial, pandas, streamlit"` runs with no error
2. Board works	Upload the Blink example	The UNO's built-in LED blinks
3. One IR sensor	Wire it to D2, upload the sketch, open Serial Monitor at 9600	Waving a block prints exactly one `E,…,PASS,1`
4. All three IR gates	Wire D3 and D4; adjust each screw to 3–5 cm	Ten blocks through each gate give a count of exactly 10
5. Buttons	Wire D5, D6, D7 to GND	Each press prints one line; Breakdown pressed twice prints DOWN then UP
6. Andon	Wire both LEDs (with resistors) and the buzzer	Green when idle; red and a beep once WIP reaches 5
7. LCD	Wire SDA/SCL to A4/A5	Both rows show and change; try 0x3F if blank
8. Bin sensor	Mount the HC-SR04, measure EMPTY_CM and PART_CM, edit and upload	The B value on the LCD matches the parts in the bin, within 1
9. Laptop link	Close Serial Monitor; run the logger, then the dashboard	Moving a block changes the dashboard within about 2 s
10. The physical line	Glue the gates, pads, bin and trays on the base board; tape every wire; fix the station tasks so Station 2 is clearly slowest	A 3-minute trial with three people runs with no miscounts
11. Count accuracy (PO1)	Three 10-minute runs; one person keeps a hand tally	Count error ≤ 2% against the hand tally

TERMS ON THIS PAGE

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

Arduino IDE: The free program used to write and upload sketches to the Arduino. *Like: A word processor for Arduino code.*

I2C (SDA, SCL): A way for chips to talk over just two wires, one for data (SDA) and one for the clock (SCL). *Like: Two people sharing one walkie-talkie channel and taking turns.*

pyserial: A Python library that reads and writes the USB serial port. *Like: The laptop's ear on the phone line.*

pandas: A Python library for working with tables of data. *Like: Excel, but driven by code.*

USB serial / COM port: The Arduino sends text to the laptop over the USB cable; the laptop sees it as a port such as COM3. *Like: A phone line between the Arduino and the laptop.*

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

IR obstacle sensor (FC-51): Shines invisible infrared light and notices when something close reflects it back. *Like: Like an automatic door that opens when you walk up to it.*

Sketch (Arduino program): The program we upload to the Arduino; it runs in a loop forever. *Like: A recipe the cook repeats again and again.*

Serial Monitor: A window in the Arduino IDE that shows the text the Arduino sends. *Like: A peephole into the phone line.*

Blocking: A station cannot pass its finished part on because the next step is full or stopped. *Like: A waiter holding plates because the table is not ready.*

5 V and GND: 5 V is the supply voltage; GND (ground) is the 0 V return path every part must also connect to. *Like: The tap and the drain: water needs both to flow.*

Andon: A signal light that tells everyone on the floor about a problem straight away. *Like: A traffic light for the line.*

Active buzzer: A buzzer with its own tone circuit, so switching on 5 V is enough to make it beep. *Like: A doorbell that rings by itself once powered.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*

I2C address (0x27 / 0x3F): The number a device answers to on the I2C wires. *Like: A house number on a shared street.*

Ultrasonic sensor (HC-SR04): Sends a click of sound too high to hear and times the echo to measure distance. *Like: A bat finding a wall by listening to its own squeak.*

Step	What to do	Check before moving on
12. Bosch data	Put `train_date.csv` and `train_numeric.csv` in `data/`; choose “Bosch sample”	Dashboard shows the three Bosch stations and a bottleneck

8. Experiments and what we will report

Run	Set-up	What we record
1. Baseline	Unbalanced line, no real cap (cap 15), 10 min, 3 repeats	Throughput, average WIP, flow time, cycle time per station, OEE proxy, scrap rate
2. Rebalanced	Part of Station 2’s task moved to Station 1, 10 min, 3 repeats	Same numbers, plus which station is now the bottleneck
3. WIP caps	Rebalanced line with caps 2, 4 and 6, 10 min each, 3 repeats	Same numbers; does the alert fire every time WIP reaches the cap (PO4)?
4. Staged bottlenecks (PO3)	One short run each: slow Station 2, breakdown, empty bin, burst of rejects	Did the dashboard point to the right cause? Target: 4 out of 4

Results so far: none, because the line is still being built. Throughput change after rebalancing: __ %. Flow-time change with a WIP cap: __ %. Little’s law gap: __ % (target ≤ 10%). We fill these only from real runs.

9. Evaluation day: set-up and live demo

Table layout (see the 3D view in section 12): the line runs left to right along the back of the table (bin, Station 1, Station 2, Station 3, good-parts tray), with the scrap tray behind Station 3. The control panel (breadboard with the Breakdown and No-material buttons, LEDs and buzzer, the LCD and the Arduino) sits in front of the line, facing the judges. The laptop with the dashboard sits at the right end, angled toward the judges.

People needed: three teammates work Stations 1, 2 and 3; the fourth presents and drives the laptop. This is the demo layout only, not a split of the project work.

Eight-minute script

1. **Set-up (1 min).** Point at each station, sensor and button and say what it measures.

TERMS ON THIS PAGE

Bosch dataset: Public data from a real Bosch production line, shared on the Kaggle website in 2016. *Like: A real factory's diary, made public.*

Bottleneck: The one step that limits how fast the whole line can make good parts. *Like: The narrow neck of a bottle that sets how fast water pours.*

Line balancing (rebalancing): Moving work between stations so they take about the same time. *Like: Sharing the bags so every hiker carries a similar load.*

Throughput: Good parts made per unit of time. *Like: Litres per minute coming out of a tap.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

Flow time (lead time): How long one part takes from entering the line to leaving it, including waiting. *Like: Your total time at the barber's, queue included.*

Cycle time: How long one station works on one part, not counting time spent waiting for work. *Like: How long the barber actually spends on one haircut.*

OEE (overall equipment effectiveness): One score: availability × performance × quality. *Like: A report-card grade combining attendance, speed and accuracy.*

Scrap rate: Scrapped parts divided by all parts made. *Like: Out of 10 rotis, how many got burnt.*

CONWIP / WIP cap: A fixed limit on how many parts may be inside the line; a new part enters only when one leaves. *Like: A club that lets one person in only when one comes out.*

Little's law: Average WIP = throughput × average flow time. *Like: More people in the queue means a longer wait if the counter's speed stays the same.*

Breadboard: A board with hidden metal strips for connecting parts by pushing in wires, no soldering. *Like: LEGO baseplate for electronics.*

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

Active buzzer: A buzzer with its own tone circuit, so switching on 5 V is enough to make it beep. *Like: A doorbell that rings by itself once powered.*

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

2. **Unbalanced run (2 min).** Release blocks. Show WIP climbing on the LCD and the chart, the pile at Station 2, and the dashboard naming Station 2 as the bottleneck. Press Breakdown for 10 s and release it, so availability drops. Reject one part so the scrap rate moves.
3. **WIP cap (1 min).** Set the cap to 4 on the slider. When WIP reaches 4 the red LED lights and the buzzer beeps, and Station 1 waits for green.
4. **Rebalance (2 min).** Move part of Station 2's task to Station 1 and run again: cycle-time bars more even, less waiting. Then show the results table from the full 10-minute runs, since a 2-minute live run is noisy.
5. **Bosch data (1 min).** Switch the data source to "Bosch sample" and show the same charts on real factory data.
6. **Close (30 s).** "OptiProductio measures count, cycle time, inventory and workflow with about ₹1,500 of sensors; rebalancing and a WIP cap changed throughput by __ % and flow time by __ %."

Bring: spare jumper wires, a spare USB cable, a charged laptop with the charger, extra blocks, tape, and a backup video of the line running on a pen drive.

10. If something goes wrong

Problem	Likely cause	Fix
One part counted twice	Sensor range too long, or the part wobbles in the gate	Turn the IR screw down to 3 cm; narrow the gate; raise IR_GAP_MS a little
A part not counted	Part passes too far from the sensor, or it is dark and reflects little	Narrow the gate; test with the real parts; use lighter blocks
LCD backlight on but no text	Contrast or address	Turn the backpack screw; change 0x27 to 0x3F
Bin count jumps around	Uneven pile, or sensor not vertical	Stack parts level; check EMPTY_CM again; make sure the sensor points straight down
"Could not open port"	Serial Monitor still open, or wrong COM port	Close Serial Monitor; check the port in Device Manager or in the Arduino IDE

TERMS ON THIS PAGE

Line balancing (rebalancing): Moving work between stations so they take about the same time. *Like: Sharing the bags so every hiker carries a similar load.*

Blocking: A station cannot pass its finished part on because the next step is full or stopped. *Like: A waiter holding plates because the table is not ready.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*

Bottleneck: The one step that limits how fast the whole line can make good parts. *Like: The narrow neck of a bottle that sets how fast water pours.*

Scrap rate: Scrapped parts divided by all parts made. *Like: Out of 10 rotis, how many got burnt.*

CONWIP / WIP cap: A fixed limit on how many parts may be inside the line; a new part enters only when one leaves. *Like: A club that lets one person in only when one comes out.*

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

Active buzzer: A buzzer with its own tone circuit, so switching on 5 V is enough to make it beep. *Like: A doorbell that rings by itself once powered.*

Cycle time: How long one station works on one part, not counting time spent waiting for work. *Like: How long the barber actually spends on one haircut.*

Bosch dataset: Public data from a real Bosch production line, shared on the Kaggle website in 2016. *Like: A real factory's diary, made public.*

Inventory: Material kept in stock waiting to be used, here the parts in the raw-material bin. *Like: The rice in your kitchen jar.*

Throughput: Good parts made per unit of time. *Like: Litres per minute coming out of a tap.*

Flow time (lead time): How long one part takes from entering the line to leaving it, including waiting. *Like: Your total time at the barber's, queue included.*

IR obstacle sensor (FC-51): Shines invisible infrared light and notices when something close reflects it back. *Like: Like an automatic door that opens when you walk up to it.*

I2C address (0x27 / 0x3F): The number a device answers to on the I2C wires. *Like: A house number on a shared street.*

USB serial / COM port: The Arduino sends text to the laptop over the USB cable; the laptop sees it as a port such as COM3. *Like: A phone line between the Arduino and the laptop.*

Serial Monitor: A window in the Arduino IDE that shows the text the Arduino sends. *Like: A peephole into the phone line.*

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

Arduino IDE: The free program used to write and upload sketches to the Arduino. *Like: A word processor for Arduino code.*

Problem	Likely cause	Fix
Dashboard says “Waiting for the Arduino…”	Logger not running, or no part moved yet	Start `serial_logger.py` first, then move one block
WIP shows a negative number	A good part passed gate 3 without passing gate 1, or Reject pressed for a part that never entered	Reset the run; make sure every part passes every gate
Cap change not reaching the Arduino	Logger started before the dashboard wrote `cap.txt`	Move the slider once more; the logger sends a cap whenever it changes

TERMS ON THIS PAGE

Arduino UNO: A small, cheap computer on one board that runs one program and reads sensors through its pins. *Like: The line's nervous system: it feels every event and reports it.*

pyserial: A Python library that reads and writes the USB serial port. *Like: The laptop's ear on the phone line.*

Blocking: A station cannot pass its finished part on because the next step is full or stopped. *Like: A waiter holding plates because the table is not ready.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*

11. References

[1] Press Information Bureau. (2024, December 23). *The MSME revolution: Transforming India's economic landscape*. Government of India. https://static.pib.gov.in/WriteReadData/specificdocs/documents/2024/dec/doc20241223476301.pdf

[2] Hwang, G., Lee, J., Park, J., & Chang, T.-W. (2017). Developing performance measurement system for Internet of Things and smart factory environment. *International Journal of Production Research, 55*(9), 2590–2602. https://doi.org/10.1080/00207543.2016.1245883

[3] Ng Corrales, L. del C., Lambán, M. P., Hernandez Korner, M. E., & Royo, J. (2020). Overall equipment effectiveness: Systematic literature review and overview of different approaches. *Applied Sciences, 10*(18), 6469. https://doi.org/10.3390/app10186469

[4] Wicaksono, A. A., Ningsih, S., & Surjati, I. (2024). Implementation of MQTT protocol on ESP32-based OEE analysis development board. *Jurnal Teknik Elektro, Emitor, 24*(2), 169–175. https://doi.org/10.23917/emitor.v24i2.3908

[5] Spearman, M. L., Woodruff, D. L., & Hopp, W. J. (1990). CONWIP: A pull alternative to kanban. *International Journal of Production Research, 28*(5), 879–894. https://doi.org/10.1080/00207549008942761

[6] Thürer, M., Li, S. S., Yang, C., Qu, T., & Huang, G. Q. (2023). Does regulating work-in-process increase throughput and reduce cycle times? An assessment by lab scale system models. In *Advances in Production Management Systems (APMS 2023)*, IFIP AICT 691 (pp. 649–661). Springer. https://doi.org/10.1007/978-3-031-43670-3_45

[7] Little, J. D. C. (2011). Little's law as viewed on its 50th anniversary. *Operations Research, 59*(3), 536–549. https://doi.org/10.1287/opre.1110.0940

[8] Mangal, A., & Kumar, N. (2017). *Using big data to enhance the Bosch production line performance: A Kaggle challenge*. arXiv. https://arxiv.org/abs/1701.00705

[9] Pettersen, J.-A., & Segerstedt, A. (2009). Restricted work-in-process: A study of differences between Kanban and CONWIP. *International Journal of Production Economics, 118*(1), 199–207.

[10] International Organization for Standardization. (2014). *Automation systems and integration: Key performance indicators (KPIs) for manufacturing operations management. Part 2* (ISO 22400-2:2014). https://www.iso.org/standard/54497.html

[11] Cordon, C. (1995). Quality defaults and work-in-process inventory. *European Journal of Operational Research, 80*(2), 240–251.

[12] Bosch. (2016). *Bosch production line performance* [Data set]. Kaggle. https://www.kaggle.com/c/bosch-production-line-performance

[13] The Patents Act, 1970, § 3(k) (India). https://www.indiacode.nic.in/handle/123456789/1392

[14] Goldratt, E. M., & Cox, J. (2004). *The goal: A process of ongoing improvement* (3rd rev. ed.). North River Press.

[15] Hopp, W. J., & Spearman, M. L. (2011). *Factory physics* (3rd ed.). Waveland Press.

[16] Silver, E. A., Pyke, D. F., & Thomas, D. J. (2017). *Inventory and production management in supply chains* (4th ed.). CRC Press.

[17] Arduino. (n.d.). *UNO R3* [Product documentation]. https://docs.arduino.cc/hardware/uno-rev3/

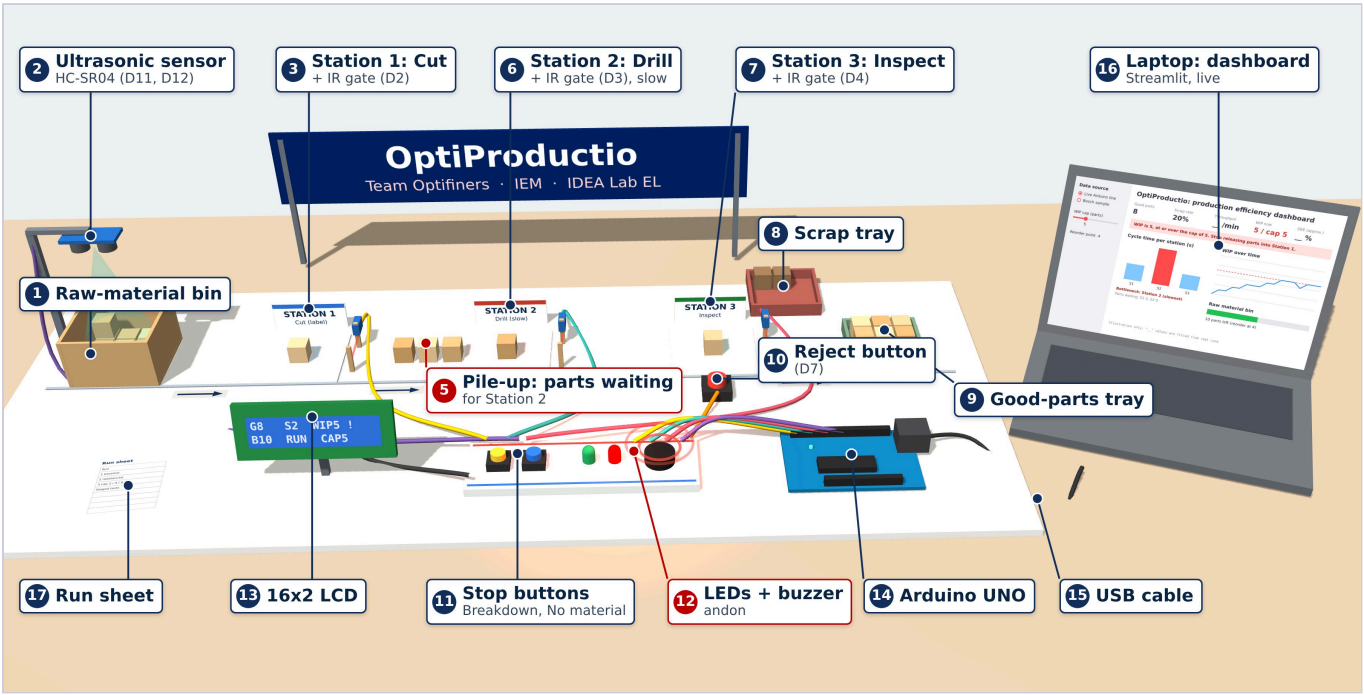
[18] ElecFreaks. (n.d.). *Ultrasonic ranging module HC-SR04* [Datasheet]. https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[19] Ohno, T. (1988). *Toyota production system: Beyond large-scale production*. Productivity Press.

[20] Streamlit. (n.d.). *Streamlit documentation*. https://docs.streamlit.io

12. 3D view: the finished line on evaluation day

These two views show how the table should look on the day of the final evaluation, with every part in its place and labelled. It is a model drawn to show the layout, not a photo, so sizes are approximate.



View 1: the whole table from the judges' side. Parts flow left to right; the control panel faces the judges; the laptop shows the dashboard.

#	Part	Its job on the day
1	Raw-material bin	Holds the blocks waiting to enter the line (inventory)
2	HC-SR04 ultrasonic sensor	Measures the bin level from above, so the dashboard can say "refill now"
3, 6, 7	Stations 1, 2 and 3, each with an IR gate at its exit	Do the task; the gate counts each part leaving (D2, D3, D4)
5	Pile-up in front of Station 2	The WIP queue the judges should see growing before we rebalance
8	Scrap tray	Where rejected parts go after the Reject press
9	Good-parts tray	Finished good parts, counted by the Station 3 gate
10	Reject button (D7)	One press per bad part
11	Breakdown and No-material buttons (D5, D6)	Log a stop and its reason; press again to restart
12, 18	Red and green LEDs, buzzer (D8, D9, D10)	The andon: green = release a part, red + beep = at the WIP cap or stopped
13	16x2 I2C LCD	Good, scrap, WIP, bin, line state and cap, at the line
14	Arduino UNO	Reads everything and sends it to the laptop
15	USB cable	Power for the Arduino and the data link to the laptop (9600 baud)
16	Laptop with the Streamlit dashboard	Runs the logger and the dashboard, angled toward the judges
17	Run sheet	Hand tally and notes for each run
19	Breadboard	Shares 5 V and GND with every part

TERMS ON THIS PAGE

Blocking: A station cannot pass its finished part on because the next step is full or stopped. *Like: A waiter holding plates because the table is not ready.*

Inventory: Material kept in stock waiting to be used, here the parts

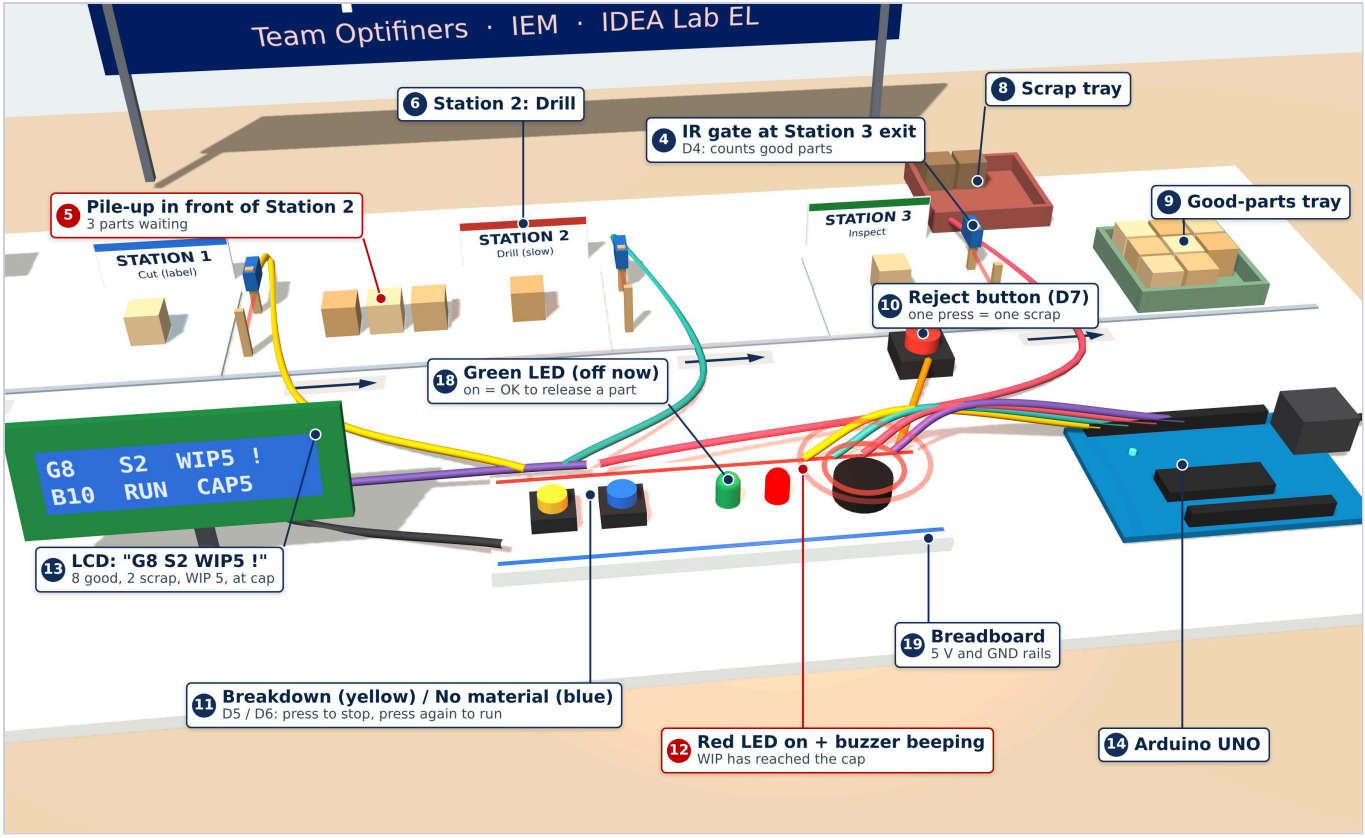
Andon: A signal light that tells everyone on the floor about a problem straight away. *Like: A traffic light for the line.*

CONWIP / WIP cap: A fixed limit on how many parts may be inside the line; a new part enters only when one leaves. *Like: A club that*

Numbers shown on the LCD and laptop screen in these views are only there to show the layout; they are not results.

TERMS ON THIS PAGE

16x2 LCD: A small screen that shows 2 rows of 16 characters. *Like: A tiny scoreboard at the line.*



View 2: close-up of the control panel and Station 3. The red LED and buzzer show the line is at its WIP cap.

TERMS ON THIS PAGE

LED and resistor: A small light; the resistor limits the current so the LED does not burn out. *Like: A tap reducer that stops a hose from bursting.*

Active buzzer: A buzzer with its own tone circuit, so switching on 5 V is enough to make it beep. *Like: A doorbell that rings by itself once powered.*

WIP (work-in-process): Parts that have entered the line but not yet left it. *Like: Clothes that are washed but not yet dried and folded.*